

OPC Code Atlas

How the repository builds a simulated recommendation world in which the logger sees users and items through distorted vectors, learns new policies from the logger's data, and grades them against the hidden truth. The two sections this revision centers on are the kinds of representation bias the study covers and the mechanisms that can correct each one. Since 4 Oct the logs are corrected: a logging bug (each user's action reused the draw that picked the user) was fixed, and every result was rerun and re-tuned on the corrected logs (`docs/simulator_fix_opc_revalidation_20261004.md`). On 5 Oct the CausE baseline was compared with the revalidated OPC at equal budget and equal capacity (`docs/cause_fair_comparison_25k.md`), and BLOB was reproduced and compared given the same supplied source (`docs/blob_controlled_integration.md`).

Repo TauRoeed/OPC, branches CRM, cause-fair and blob-controlled-integration Code at de1d335 (pushed), 5 Oct 2026

Functions are named, not line-numbered Paths are relative to the repo root

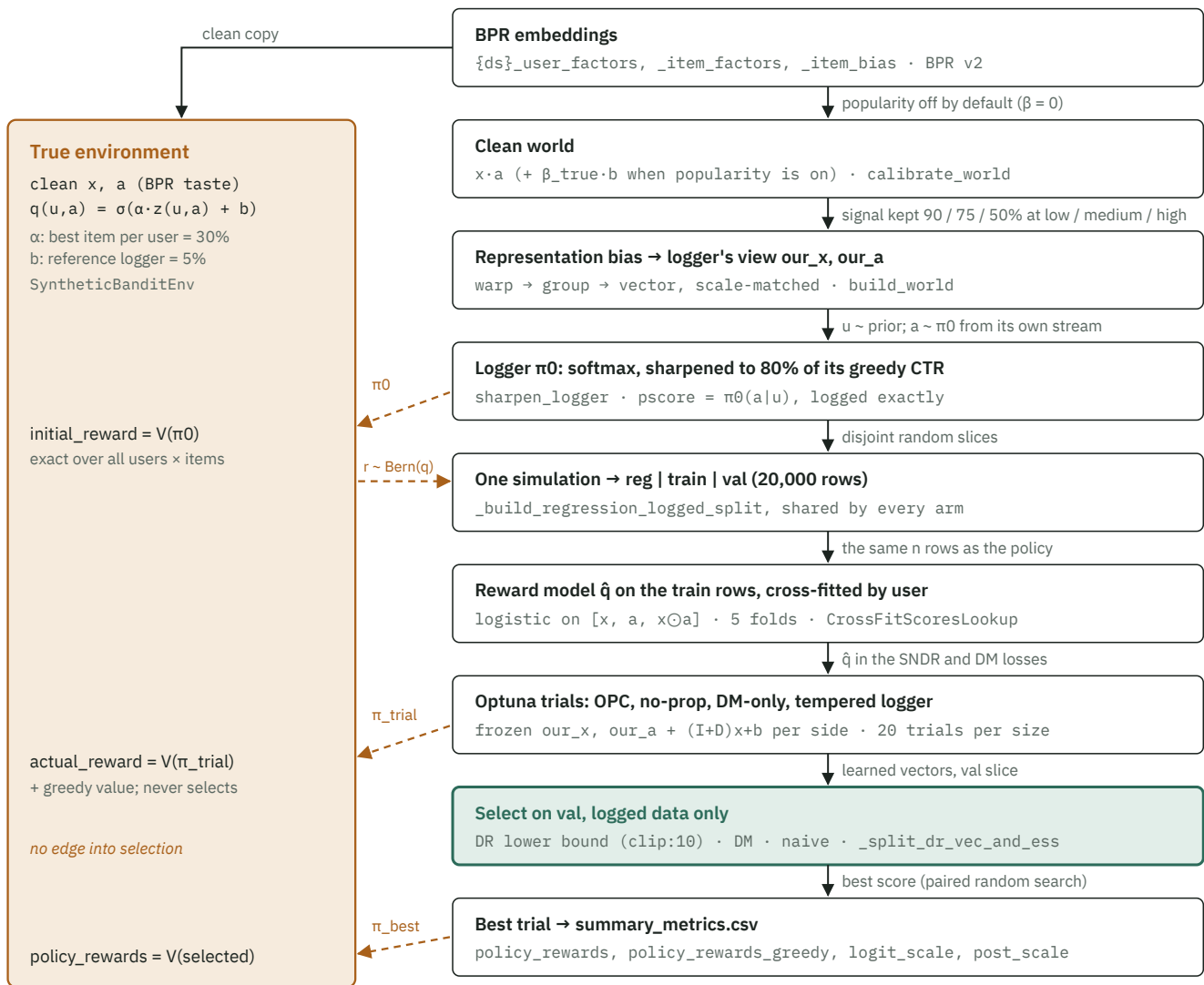
THE RESEARCH QUESTION

Can off-policy learning undo a logger's representation bias?

A deployed recommender scores items with learned user and item vectors. Those vectors are wrong in structured ways: a global misalignment of taste axes, whole user segments misread, individual tastes never learned. The logs it collects inherit those errors. The study asks which of these distortions a policy learned from the logs can correct, with which mechanism, how much data it takes, and whether logged propensities add anything beyond a good reward model. The simulator makes every distortion a dial and computes every policy's true click rate exactly.

The pipeline at a glance

A study is a grid of *conditions* (dataset × bias configuration × seed, run at several training sizes). Each condition runs the chain below for every arm. The design rests on the wall between the two columns: the true environment produces the rewards and grades the results, and nothing that chooses a policy ever reads it.



Solid arrows are the learning path; dashed amber arrows touch the hidden truth. The truth supplies the logged rewards and grades three things (the logger, every trial, the selected policy), but nothing flows from it into selection (green), which reads only the validation rows of the log.

FOCUS 1

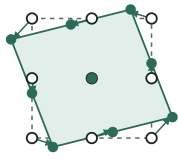
Representation bias: the distortions we cover

The truth is BPR's clean user and item vectors. The logger, the reward model and every learned policy see biased copies instead. Three kinds of bias are mixed in, in this order, on both sides (users and items), each rescaled to the clean vectors' size so a level means a share of signal replaced and never an inflation of vector length:

$\text{biased} = (1 - \epsilon) \cdot \text{current} + \epsilon \cdot \text{target}$, applied for warp, then group, then vector, and rescaled to the clean RMS after each step (`utils/representation_bias.py · SideBias.biased`).

Warp

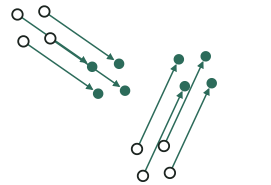
one linear map for the whole side



Taste axes misaligned for everyone:
a stale or globally mis-trained model
target = $x \cdot W$, $W \sim N(0, 1)^{(d \times d)}$

Group

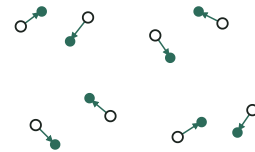
one offset per group of users or items



segment A segment B
Whole segments' tastes misread
target = $\text{template}[\text{group}(x)]$, k-means

Vector

independent noise per user and item



Individual tastes never learned:
sparse users, cold items
target = $N(0, I)$ per vector

Open circles are clean vectors, filled green dots the logger's biased view. A warp moves everyone by one shared map, so its structure is global. A group offset moves a whole segment together. Vector noise moves each user or item on its own, so no shared structure is left to learn from.

Levels, configurations and calibration

The strength ϵ of each type is calibrated per dataset so that **all three together** keep 90%, 75% or 50% of the taste signal at low, medium and high bias. Signal kept is the mean, over users, of the correlation across items between biased and clean taste scores. A single type at level L alone keeps κ_L of the signal (about 0.97, 0.94 and 0.88). A configuration names the three levels, for example `high/none/none` for warp only.

CONFIGURATION	WHAT IT ISOLATES	LOGGER'S GREEDY RANKING REACHES (SHARE OF THE 30% CEILING, ML / KUAIRAND / ANIME)	COVERED IN
<code>none</code>	control: no representation bias, only the logger's exploration	100%	dev grid, final report
<code>low · medium · high</code>	all three types together, 90 / 75 / 50% signal kept	80–89% · 59–73% · 38–47%	Test 2 (medium, high); final report (all)
<code>high/none/none</code>	warp only	73% / 82% / 73%	dev grid, final report
<code>none/high/none</code>	group only	77% / 87% / 79%	dev grid, final report
<code>none/none/high</code>	vector only	72% / 80% / —	dev grid, final report
<code>--pop-strength</code> , <code>--logger-pop-strength</code>	popularity: the truth weighs BPR's item bias b by β_{true} , the logger by β_{log} (over-exposure when $\beta_{\text{log}} > \beta_{\text{true}}$)	—	PLANNED RUNS (the dial exists)

The logger that collects the data

A softmax over the biased scores, sharpened per condition until it earns **80% of its own greedy CTR** (`--logger-greedy-share 0.8`, `sharpen_logger`). At that setting its temperature is about $T/4.2$ and it spreads over 110–260 items per user. It mostly exploits its ranking, which the bias has made wrong, and explores near the top of it. The spread temperature T , at which the clean logger covers half the catalog, only calibrates the click model. `off` restores the older near-uniform logger, which earned about 26% of its greedy value.

FOCUS 2

Correction mechanisms

A learned policy can fight the bias at three points: in what the policy can represent, in the objective it is trained on, and in how one of many trained candidates is chosen. Chips mark what exists today and what is planned.

1 · The policy class

What distortion of the vectors can the learned policy undo?

Linear correction

DEFAULT

(I + D)·x + b on the user side and on the item side, starting exactly at the logger (D = 0, b = 0). It can invert a warp exactly and remove a shared shift.

```
--policy-transform linear ·  
GlobalLinearCorrection
```

Linear + MLP

OPTION

The linear correction plus a zero-initialized MLP residual on the raw vector, with no LayerNorm and no dropout. Can bend the space nonlinearly, for example around group boundaries.

```
--policy-transform linear+mlp
```

MLP (older)

LEGACY

x + MLP(LayerNorm(x)), random start. Kept to reproduce earlier runs.

```
--policy-transform mlp
```

Sharpness: logit scale, post-hoc tempering

OPTIONS

A learnable s in softmax(s·u·a/T), or s chosen after training on validation from 0.25–16. These change how concentrated the policy is, never its ranking. The study learns s (re-checked on corrected logs: a fixed scale loses 0.2–1.3 points, and tempering on top does not change the selected policy).

```
--learn-logit-scale · --post-temper
```

Learnable popularity weight

AUTO

With a popularity column, the policy learns its own weight on b, starting at the logger's. This corrects over- or under-exposure.

```
CFModel(pop_weight)
```

Per-group and per-item offsets

PLANNED

A learned offset per user group (k-means on the biased vectors) and per item, pulled toward zero with strength shrinking as the group's or item's logged data grow (partial pooling). Aimed at group bias and at the part of vector bias that has enough data behind it.

2 · The training signal

How is a biased log turned into an objective?

Propensity correction (OPC)

DEFAULT ARM

The reward model's value of the policy plus an importance-weighted correction on the logged rows. This removes the logger's exposure bias where the logger explored. The working development default is DR, differentiated directly, with harmonic:0.1 weights; selection stays clip:10 with the 95% lower bound. Set on 27 Sep (f5cade9) and re-tuned on the corrected logs on 4 Oct. It is the working method, not the final paper choice. Exact SNDR (the gradient of the full-data ratio) is the legitimate self-normalized alternative; on corrected logs it adds nothing over DR with harmonic weights. Legacy minibatch SNDR, global SNDR (a stop-gradient surrogate) and the log trick remain for reproducibility only.

```
--policy-losses dr (default) | sn dr  
· --sn-scope exact|batch|global · -  
-opc-gradient direct (default) |  
log-trick
```

Importance-weight transform

OPTIONS

The options are none (raw DR), clip:M, shrink:λ (Su et al. 2020) and harmonic:λ (Metelli et al. 2021): w/(1 - λ + λw), at most 1/λ, and trained by its direct gradient. Under the log trick, a transform g optimizes H(w) = ∫ g(t)/t dt rather than the named estimate; for shrink:100 that is 10·arctan(w/10). On corrected logs every regularized transform beats raw DR on average (the best by about 0.3–0.5 points per cell), and the best amount of regularization falls with n. harmonic 0.1–0.3 lead; harmonic:0.1 is the most robust across sizes and stays the default. shrink:100 is within noise of it and is the robustness alternative; none (raw DR) is the reference.

```
--train-weights (default  
harmonic:0.1) ·  
docs/training_losses.md §3.4, §9
```

Reward model alone (DM-only)

BASELINE

Maximizes the policy's value under $\hat{q}$. It reaches items the logger never showed if $\hat{q}$ extrapolates well; no propensities.

```
dm loss · --methods dm
```

Logged clicks as they are (no-propensity)

BASELINE

Raises the probability of clicked logged items. It inherits the logger's exposure (it favors $\pi_0 \times q$) and stays near the logger.

3 · Selection

Which of 20 trained candidates is deployed?

DR lower confidence bound

OPC

Mean - t·SE of the doubly robust estimate on 20,000 validation rows, weights clipped at 10. On corrected logs it picks within 0.03–0.11 points of the best of 20 trials; raw-weight selection loses up to 0.8 points.

```
--select-weights clip:10 · --  
optuna-selection ci_low
```

Reward-model score

DM-ONLY

The DR score with every weight 0. select_estimator="dm"

Naive score

NO-PROPENSITY

Mean of r·π on validation.

Tempered logger

BASELINE

No training at all: the logger's logits × s, with s chosen by the DR score. It measures how much of any gain is sharpening alone.

```
--methods tempered_logger
```

Every rule, after the fact

LOGGED

Every trial of every arm carries a selection score per logged rule (19 in the revalidation runs: 18 weight transforms and DM), so any selection rule can be replayed on any training rule without new runs.

```
--log-select-weights
```

naive loss

The reward model $\hat{q}$ DEFAULT

Logistic regression on $[x, a, x \odot a]$ of the biased vectors, so rankings can differ between users. It is fit on the same n training rows as the policy and cross-fitted by user (5 folds).

```
--reward-features interaction · --
reward-data train · --crossfit-
folds 5
```

Bilinear reward model $x^T W a$ PLANNED

A full (ridge or low-rank) bilinear form represents a warp exactly; $x \odot a$ captures only its diagonal.

Cluster-level weights PLANNED

Importance weights over item clusters (MIPS, OffCEM) or a two-stage policy (POTEC), for catalogs where item-level weights collapse (msd, lastfm).

FOCUS 3

Which mechanism can fix which bias

Each cell rates a correction mechanism against a bias type, by construction and by the oracle bound. The oracle trains the policy class directly on the true click probabilities, with unlimited data (ml, high level of each type alone).

BIAS TYPE	LINEAR CORRECTION	GROUP / ITEM OFFSETS	REWARD MODEL $x \odot a$	BILINEAR REWARD MODEL	PROPENSITY CORRECTION	ORACLE BOUND, ML (GREEDY LOGGER $\rightarrow$ LINEAR $\rightarrow$ + GROUP OFFSETS)
Warp	exact	not needed	diagonal only	exact · planned	where π_0 explored	22.1 $\rightarrow$ 29.7 $\rightarrow$ 29.2 (ceiling 30.4)
Group	shared shift only	targets it · planned	partial	partial	where π_0 explored	23.5 $\rightarrow$ 28.2 $\rightarrow$ 28.9
Vector	de-noising only	items with data · planned	no structure	no structure	where π_0 explored	21.8 $\rightarrow$ 24.9 $\rightarrow$ 26.3
Popularity exposure	—	—	sees b	sees b	removes exposure	planned runs; policy learns its own weight on b
Logger sharpness (not a bias)	logit scale or post-hoc tempering; the tempered-logger baseline measures this part				noisy for flat loggers	sharpening alone: logger 15.5 $\rightarrow$ 19.3 (medium)

Warp is almost fully fixable by the linear class (7.6 of 8.3 lost points back), group bias mostly, vector noise only partly. Propensities help wherever the logger's exploration reached, so a sharper logger helps less at small data (see Findings). The dev grid measures these cells with actual learning: linear vs linear+MLP vs MLP, with and without post-hoc tempering, for each type alone.

The four arms

All arms get the same splits, the same reward model, the same search budget (20 trials per size), the same seed and, where they apply, the same weight rules. What differs is how each turns the log into an objective and a selection score.

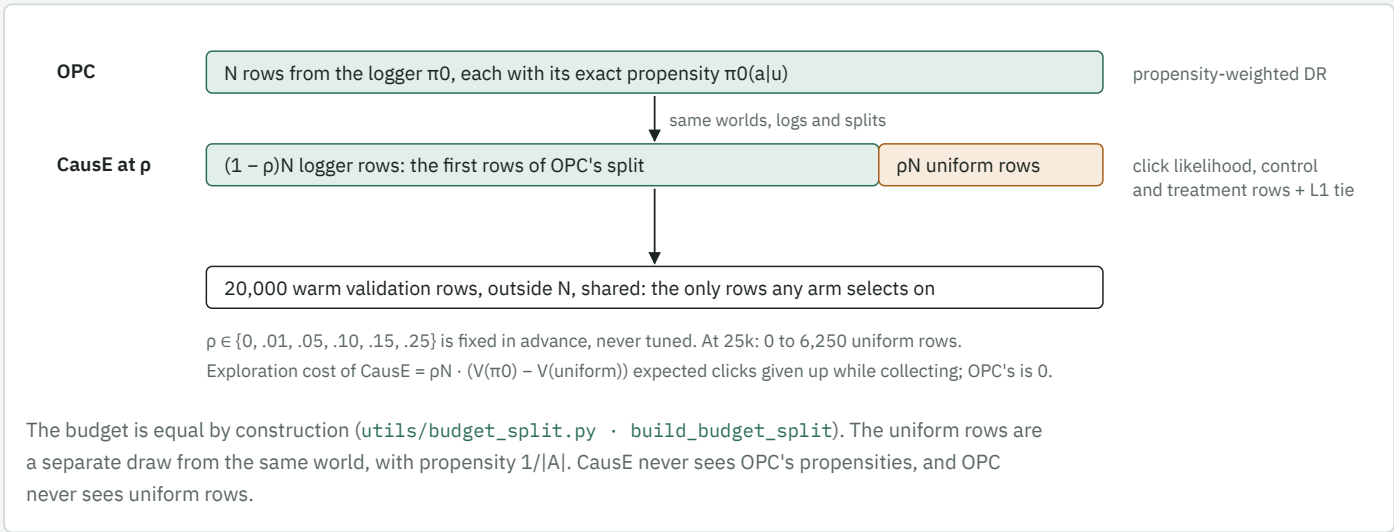
ARM	TRAINING OBJECTIVE	SELECTION SCORE	WHAT IT ISOLATES
OPC DEFAULT	DM($\hat{q}$) + importance-weighted correction: DR, direct gradient and harmonic:0.1 weights (working default); shrink:100 is the standard comparison; legacy and global SNDR and the log trick are kept for reproducibility	DR lower bound, clip:10	the method: reward model + propensities

ARM	TRAINING OBJECTIVE	SELECTION SCORE	WHAT IT ISOLATES
No-propensity DEFAULT	naive: $-\text{mean } r \cdot \pi(a x)$	naive mean $r \cdot \pi$	learning from clicks without correcting exposure
DM-only OPT-IN	$-\sum \pi(a x) \cdot \hat{q}(x,a)$	$\hat{q}$ (all weights 0)	what a good reward model does alone
Tempered logger OPT-IN	none: logger's logits $\times s$, s in 0.5–64	DR lower bound	sharpening without any correction

PRIOR-WORK BASELINE

CausE: randomized traffic instead of propensities

CausE (Bonner & Vasile, RecSys 2018) learns a click model from the logged traffic plus a small uniform-random sample, with a control and a treatment copy of every item tied by an L1 discrepancy. The comparison asks one question. Given the same source representation and the same total budget N of target interactions, is it better to spend pN of them on uniform traffic (CausE) or to keep all N from the logger and use its propensities (OPC)? Development stage, 25k, branch `cause-fair` (`docs/cause_fair_comparison_25k.md`).



VARIANT	STARTS FROM	CAPACITY	STRUCTURAL CEILING	ROLE
Native CausE (prod-C, prod-T, avg)	Xavier-random vectors; never sees the source vectors	free user vectors, two item tables, per-row biases	the target-best value (its class contains the true click model)	the published method as it is (M5 rows, reused)
CausE-warm (C, T)	OPC's source vectors: $U \leftarrow x$, both item tables $\leftarrow a$; intercept 0 or the base rate	the native capacity, everything trainable	the target-best value	the source representation's effect at CausE's capacity
CausE-capacity-matched (C, T)	the frozen source vectors with OPC's maps at zero	$(I + D)x + b$ per side: users, control items, treatment items	OPC's Stage 1 linear-repair oracle	random vs propensity traffic at equal capacity; its greedy family equals OPC's (a test checks equal logits)

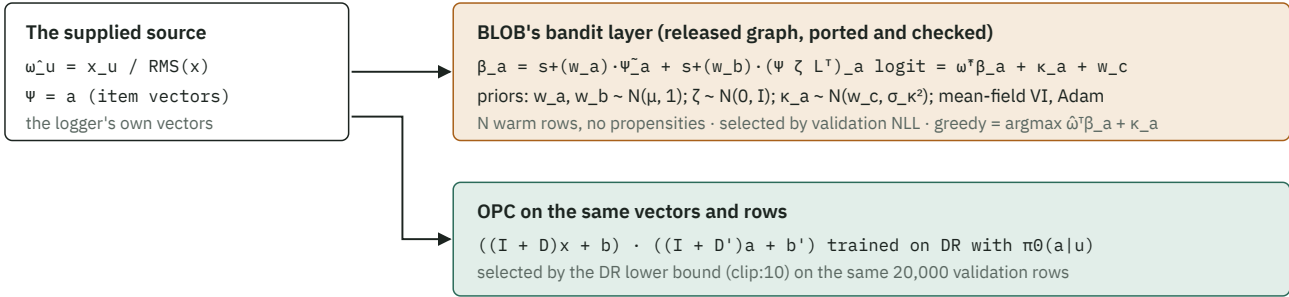
Every variant is trained by CausE's released objective (mean cross-entropy, L2, the tie; momentum with linear decay) and selected by validation NLL. The greedy (ranking) value is primary. For the stochastic value a click model's softmax has no natural temperature, so each selected model gets the tempered logger's mechanism: logits $\times s$, with s chosen by the DR lower bound (clip:10) on validation and $\hat{q}$ fit on CausE's own N rows. Search spaces were fixed on tuning seeds 200/201 before the main grid; 20 trials per variant, p and world, as OPC has per world.

```
models/cause.py (CausEModel, warm_start_, CausELinModel, batched training) ·
training/cause_trials.py · cause_trainer_trial(--methods cause --cause-family
native|warm|cap) · training/analyze_cause_fair.py (tuning and comparison tables, figures).
```

PRIOR-WORK BASELINE

BLOB: a Bayesian bandit layer on the source representation

BLOB (Sakhi, Bonner, Rohde & Vasile, KDD 2020) learns user and item representations from organic sessions, then fits a Bayesian click model on bandit feedback whose prior ties it to those representations. It never uses propensities. The controlled question: given *the same* source representation and the same N target interactions, how does BLOB compare with the plain likelihood learner, DM-only and OPC? Development stage, 25k, branch `blob-controlled-integration` (`docs/blob_controlled_integration.md`).



BLOB-supplied-source replaces BLOB's organic model by the logger's own vectors, so BLOB gets no source information beyond OPC's. Both rank items with a $K \times K$ bilinear map of the same vectors; BLOB adds per-item intercepts (with a tight prior) and has no per-user terms.

ARM (25K, THE SAME SOURCE AND ROWS)	BIASED WORLDS (24)	NO BIAS (6)	WARP	GROUP	VECTOR	COMBINED
BLOB-NQ (supplied source, no propensities)	+1.42 [+0.85, +1.98]	-0.17	+1.58	+0.70	+0.84	+2.56
BLOB-MNQ	+0.94 [+0.53, +1.35]	-0.11	+0.71	+0.54	+0.75	+1.75
CausE-cap-C, $\rho = 0$ (plain likelihood, OPC's class)	+3.09 [+2.10, +4.08]	-0.55	+4.23	+1.22	+1.03	+5.89
OPC (harmonic:0.1)	+2.69 [+1.84, +3.53]	-0.61	+3.03	+1.32	+0.96	+5.42
DM-only (own range)	+2.15 [+1.30, +3.01]	-0.55	+2.09	+0.81	+0.64	+5.07

Greedy value minus the logger's greedy value, CTR points (mean over worlds; 95% CI for the pooled biased worlds). BLOB-NQ – CausE-cap: -1.67 [-2.29, -1.06], higher in 2 of 24 biased worlds; BLOB-NQ – OPC: -1.27 [-1.77, -0.76], 2 of 24; without bias BLOB-NQ – CausE-cap: +0.38 [+0.20, +0.56], 6 of 6.

BLOB's released code reproduces its paper's Table 3 (every BLO and BLOB entry within 0.02 CTR points). Here BLOB-supplied-source gets the logger's own vectors as its organic representation and the same 25k warm rows; its search space came from a pre-registered rule on seeds 200/201, with one supplementary round; selection is by validation NLL. Ceilings come from truth-trained class oracles: the value oracle (the class's best policy) and the likelihood oracle (the infinite-data likelihood fit under the logger).

`models/blob.py` (`BlobBandit`, `BlobBanditBatch`, TF1 Adam; checked step by step against the unmodified TensorFlow graph) · `training/blob_trials.py` · `blob_trainer_trial` (`--methods blob --blob-families nq mnq`) · `training/class_oracles.py` (truth-trained value and likelihood oracles per score class) · `training/policy_diagnostics.py` (pick diagnostics) · `training/analyze_blob.py` (tuning rule, comparison tables and figures) · `scripts/blob_reference/` (the TF fixture and the Table 3 reproduction).

Step by step through one condition

Order of execution inside `training/run_full_study.py` · `_run_condition` and `training/trainer_trials.py` · `regression_trainer_trial`. The parallel and H1 runners call the same function.

#	WHAT HAPPENS	WHERE
1	Seed and pin. Deterministic kernels, fixed CPU threads, <code>seed_everything(seed)</code> ; every later stream uses <code>derive_seed(seed, labels...)</code> .	<code>utils/seeding.py</code>
2	Build the world. Calibrate once per dataset and seed (bias strengths ϵ , spread temperature T, click model α and b), then for this configuration mix the bias into the logger's view and sharpen the logger.	<code>calibrate_world</code> · <code>build_world</code> · <code>sharpen_logger</code>

#	WHAT HAPPENS	WHERE
3	Simulate and split. One log per train size (reg + train + 20,000 validation rows), cut into disjoint slices, cached and shared by every arm.	<code>LazyRegressionSplitCache</code>
4	Reward model. Fit on each size's training rows. In the default mode it is cross-fitted by user in 5 folds: each fold's model sees the other folds' rows. The external 50k-row slice is still drawn, so splits match the older mode.	<code>fit_shared_regression_bundle</code> · <code>_crossfit_bundles</code>
5	Initial reward. Exact true value of the logger, and of its greedy ranking.	<code>_dataset_log_constants</code> · <code>calc_greedy_reward</code>
6	Optuna trials per arm. Each trial reseeds, samples lr, epochs, batch size and decay, and builds a <code>CFModel</code> on the frozen biased vectors with the chosen correction. It then trains with the arm's loss, where the training loss reads the cross-fitted $\hat{q}$.	<code>regression_trainer_trial</code> · <code>CFModel</code> · <code>train</code>
7	Optional post-hoc tempering of the trained policy (logits × s chosen on validation).	<code>_post_temper</code>
8	Score on validation (logged data only) with the arm's selection score; Optuna maximizes the lower bound. The trial also logs its true value, its greedy value, the weight diagnostics and the 14 re-selection scores. None of these is used to select.	<code>_split_dr_vec_and_ess</code> · <code>_selection_score_variants</code>
9	Best trial kept (no retrain). With TPE (the default), its hyperparameters are enqueued as the first trial at the next train size. With <code>--sampler random</code> there is no warm start, so every run of the grid trains the same configurations with the same seeds (paired comparisons).	<code>study.enqueue_trial</code>
10	Write outputs for every arm.	<code>_finalize_summary_df</code>

Key data objects

NAME	WHAT IT IS	USED FOR
<code>emb_x</code> , <code>emb_a</code> TRUTH	clean vectors (with a popularity column when popularity is on)	the click model; the oracle reward model
<code>our_x</code> , <code>our_a</code>	the biased view	the logger, the policy's frozen starting vectors, the reward model's features
<code>policy_temperature</code>	the logger's sharpened temperature T/f	the logger and every learned policy (which start at the logger)
<code>pscore</code>	$\pi_0(a u)$ of the logged action, exact	importance weights
<code>q_x_a</code> TRUTH	dense true $q(u,a)$ when $\leq 25M$ cells	exact values, reward draws
<code>RegressionScoresLookup</code> , <code>CrossFitScoresLookup</code>	$\hat{q}$ rows for batches: dense or closed form; cross-fitted per user	DM terms in training and selection
<code>world</code> record	every calibrated number: ϵ per type and level, signal kept, α , b, T, the logger's sharpness, effective items, greedy CTR	<code>run_meta.json</code> , summary columns

Entry points and the main flags

COMMAND	PURPOSE
<code>python -m BPR.generate_artifacts</code> · <code>BPR.evaluate</code>	fit BPR v2 per dataset (factors + item bias); test-set Recall, NDCG and MPR
<code>python -m training.run_full_study(_parallel)</code>	the study; the parallel runner pools conditions across workers and GPUs
<code>python -m training.characterize_world</code>	print a dataset's calibrated world per bias configuration
<code>python -m training.oracle_repair</code>	the oracle repair bound: the learner's policy class fit to the true click model (structural recoverability)
<code>python -m training.oracle_validation</code>	checks the oracle bounds: the same class, past the old learning-rate edge, with a longer budget
<code>python -m training.analyze_recoverability</code>	recoverability tables: <code>stage1</code> (oracle) and <code>stage2</code> (learned arms as a fraction of the oracle repair)
<code>python -m training.representation_report</code>	every figure and table of the experimental report, from the committed summaries
<code>python -m training.class_oracles</code>	truth-trained value and infinite-data likelihood oracles for the affine-bilinear, BLOB and bilinear score classes
<code>python -m training.policy_diagnostics</code>	pick diagnostics of saved policies: where they recommend relative to the logger, the value there, a click model's error by logging propensity
<code>python -m training.analyze_blob</code>	<code>tune</code> (the BLOB tuning rule) and <code>compare</code> (the BLOB comparison's tables and figures)
<code>python -m training.run_h1_study</code>	the H1 grid (bounded-error reward model); still on the external reward model

FLAG	DEFAULT	CONTROLS
<code>--bias-configs</code>	low medium high	levels per type, e.g. <code>high/none/none</code>
<code>--logger-greedy-share</code>	0.8	logger sharpness (<code>off</code> = spread logger)
<code>--pop-strength</code> , <code>--logger-pop-strength</code>	0, = truth	popularity in the truth and in the logger
<code>--methods</code>	opc no_propensity	add <code>dm</code> , <code>tempered_logger</code>
<code>--policy-transform</code>	linear	linear, linear+mlp, mlp
<code>--learn-logit-scale</code> , <code>--post-temper</code>	off	sharpness learned in training, or chosen after training
<code>--reward-features</code>	interaction	[x, a, x⊙a] or concat [x, a]
<code>--reward-data</code> , <code>--crossfit-folds</code>	train, 5	q̂ on the policy's own rows, cross-fitted; <code>external</code> = separate 50k slice
<code>--policy-losses</code> , <code>--sn-scope</code>	dr, batch	OPC objective: dr (the default, revalidated 4 Oct); sndr with the exact scope (the gradient of the full-data SNDR ratio, direct gradient only), or the legacy batch / global surrogates (reproduction only)
<code>--lr-range</code> , <code>--epochs-range</code> , <code>--lr-decay-range</code>	1e-4 1e-3, 5 25, 0.8 1	the trained arms' shared search space; the revalidated study uses <code>--lr-range 1e-4 2e-3 --epochs-range 5 30</code>
<code>--weight-decay-range</code>	off	AdamW decay toward the logger, drawn per trial from its own stream (random sampler only)
<code>--opc-gradient</code>	direct	how OPC's loss is differentiated; <code>log-trick</code> reproduces the runs before f5cade9
<code>--train-weights</code> , <code>--select-weights</code>	harmonic:0.1, clip:10	weight transforms in training and in selection: none, clip:M, shrink:λ, harmonic:λ
<code>--sampler</code> , <code>--stage</code>	tpe, development	random = paired replay of identical configurations; the run's purpose, recorded with the results
<code>--val-size</code>	20000	fixed validation rows (<code>0</code> = fraction rule)
<code>--methods blob</code> , <code>--blob-families</code>	off, nq	BLOB-supplied-source (opt-in; needs <code>--sampler random</code>): one row per family, <code>blob_nq</code> , <code>blob_mnq</code>
<code>--blob-lr-range</code> , <code>--blob-epochs</code> , <code>--blob-wa-m</code> , <code>--blob-wb-m</code> , <code>--blob-kappa-s</code>	released values	BLOB's search space: lr, epochs, the prior means of w_a and w_b, the intercepts' prior std (searched when several are given)
<code>--blob-seed-tag</code> , <code>--blob-pick-diagnostics</code>	off	an independent set of trials (a supplementary tuning run); every trial's pick diagnostics
<code>--save-policies</code>	off	save each arm's selected policy in its condition folder, for <code>training.policy_diagnostics</code>

The runs before 26 Sep reproduce exactly with `--reward-features concat --train-weights none --select-weights clip:1 --policy-transform mlp --logger-greedy-share off --reward-data external --val-size 0` (checked end to end against ce827f5).

Outputs and what the columns mean

COLUMN	MEANING
<code>policy_rewards</code> TRUTH	true CTR of the selected policy; at train size 0, the logger's
<code>policy_rewards_greedy</code> , <code>actual_reward_greedy</code> TRUTH	true CTR of the policy's top item per user: rankings compared without sharpness
<code>actual_reward</code> TRUTH	each trial's true CTR (never used to select)
<code>value</code> , <code>r_hat</code>	each trial's selection score (lower bound) and point estimate
<code>sel_ci_low[spec]</code> , <code>sel_r_hat[spec]</code>	re-selection scores under 14 rules, every arm
<code>ess</code> , <code>ess_raw</code> , <code>diag_*</code>	effective sample size of the transformed and raw weights; weight profile; DM vs correction split
<code>logit_scale</code> , <code>post_scale</code>	learned and post-hoc sharpness factors
<code>trial_time_s</code> , <code>diverged</code> , <code>param_weight_decay</code>	each trial's wall time; whether training stopped at a non-finite gradient (the trial keeps its last finite parameters); its AdamW decay
<code>code_commit</code> , <code>search_space</code> (run_meta.json, manifest)	the commit and dirty flag of the code that ran; the search space
<code>logger_greedy_share</code> , <code>logger_sharpness</code> , <code>logger_greedy_ctr</code>	the logger of the condition
<code>reward_data</code> , <code>crossfit_folds</code> , <code>post_temper</code> , ...	the settings of the run, one column each

Condition folders name every non-default setting, e.g.
`dataset=m1__bias=medium__ctr=0.05__seed=100__qhat=train__cf=5__val=20000`.

Modules

utils/

- `representation_bias.py`: calibration, the three bias types, logger sharpness, world record
- `simulation_utils.py`: click model, `generate_dataset`, logging, exact and greedy values
- `importance_weights.py`: none, clip, shrink, harmonic, dm; ESS
- `policies.py`, `seeding.py`, `noise_snr.py`, `provenance.py` (the code commit of a run)
- `budget_split.py`: CausE's fixed budget, $(1 - p)N$ logger rows + pN uniform rows

models/

- `models.py`: CFModel (logit scale, popularity weight), corrections, `RegressionModel`
- `custom_losses.py`: snr (batch or global scope), dr, dm, ipw, naive, kl, crm; log-trick or direct
- `estimators.py`: post-hoc DM, DR, SNIPW, SNDR (clip, shrink, harmonic)
- `cause.py`: CausE (prod and avg layouts, the released objective, checked against the TensorFlow graph), `warm_start`, `CausELinModel`, batched trials
- `blob.py`: BLOB's bandit layer (NQ and MNQ, the released priors, Ψ normalization, noise term and TF1 Adam; checked step by step against the unmodified TensorFlow graph), batched trials

training/

- `trainer_trials.py`: splits, reward models, cross-fitting, trials, selection, tempering
- `run_full_study(_parallel).py`: conditions, arms, flags, outputs
- `oracle_repair.py`: the oracle repair bound; `oracle_validation.py`: its check; `analyze_recoverability.py`: recoverability, per-dataset and gap tables
- `memory_budget.py`: workers per GPU from each condition's estimated peak and the free memory
- `representation_report.py`: the experimental report's figures and source tables
- `cause_trials.py`: the CausE arm (native, CausE-warm, CausE-capacity-matched; fixed-budget data, seeded search, NLL selection, the fair tempering); `analyze_cause_comparison.py`, `analyze_cause_fair.py`: the M5 and the fair comparison's tables and figures
- `blob_trials.py`: the BLOB-supplied-source arm (the logger's vectors as BLOB's source, seeded batched search, NLL selection, the fair tempering); `class_oracles.py`: truth-trained oracles per score class and objective; `policy_diagnostics.py`: pick diagnostics of saved policies; `analyze_blob.py`: the BLOB tuning rule and comparison
- `logging_coupling_diagnostic.py`: the exact distribution the pre-fix logger generated; `analyze_revalidation.py`, `revalidation_phase3.py`, `revalidation_compare.py`, `revalidation_report.py`: the revalidation's tuning analysis, summaries, old-vs-new pairing and report
- `training_utils.py`, `characterize_world.py`, `run_h1_study.py`

BPR/

- `bpr.py`, `generate_artifacts.py`: BPR v2 with item bias, numpy trainer
- `evaluate.py`: test-set metrics on CPU and GPU
- `bpr_config.py`: per-dataset recipes and the default six datasets

Tests

589 tests pass with the GPU; 559 pass and 11 are skipped with it hidden (at `de1d335`, branch `blob-controlled-integration`). Every change since 25 Sep was also checked bit for bit against the previous commit with the new option off. New on 5 Oct, with BLOB:

- the BLOB port replays the authors' unmodified TensorFlow graph step by step in five cases (losses to $2e-5$, variables to $1e-4$, the released initialization exactly); the released Ψ normalization aliases the prior mean; the fixture's provenance is the unmodified release;
- batched BLOB trials equal separate runs, and the sync-free training loop is bit-identical to the first one (a diverging trial included);
- the BLOB arm end to end, deterministic, with the supplied source at unit RMS;
- every class oracle starts at the logger; the ranking vectors and the click offset reproduce the scores;
- the pick diagnostics against dense numpy; every arm's saved policy (OPC, CausE-cap, BLOB) reproduces its own greedy value;
- the BLOB tuning rule on synthetic trials (the simulated protocol, the decision, the edge rule).

New on 5 Oct, with the CausE comparison:

- CausE's released objective against the unmodified TensorFlow graph; batched trials equal their single runs;
- CausE-warm starts exactly at the source vectors; CausE-capacity-matched gives the same logits as OPC's policy for the same maps ($\alpha = s/T$); at $p = 0$ only the symmetric tie moves the treatment side;
- tempering only the selected trial reproduces tempering every trial; `--skip-completed` recognizes every CausE family's labels;
- the fixed budget: $N_c + N_t = N$, the warm rows are a prefix of OPC's split, the uniform rows a separate draw;
- the fair comparison's analysis: the resampled k-trial protocol equals direct selection when k covers every trial; paired contrasts by world.

New on 4 Oct, with the simulator fix:

- logged rows are draws from the propensities they record: per fixed user (chi-square at three temperatures, with and without a uniform mix), jointly over users and actions, and through the IPS identities $E[\pi/\text{pscore}] = 1$ and $E[\pi/\text{pscore} \cdot r] = V(\pi)$. 13 of these 15 tests fail on the pre-fix code;
- the smallest failing case: two users, two actions, $\pi_0 = (\frac{1}{2}, \frac{1}{2})$; with the coupled streams user 0 always got action 0;
- a sharper logger is more concentrated in the population and in the logs (repeat rate $= \Sigma \pi_0^2$);
- the search space: the default ranges reproduce the run without one, a new range changes only its own draws, and AdamW decay comes from its own stream so trials stay paired;
- a diverging trial keeps its last finite parameters; exact SNDR equals the literal full-data ratio's gradient;
- the revalidation report rebuilds from the committed summaries alone, and its old-vs-new numbers are paired per-world differences.

Earlier (27 Sep):

- each objective variant's gradient against literal full-data autograd;
- equal row weights across the DataLoader's short final batch;
- identical replayed configurations;
- log-trick and direct gradients for every transform;
- the harmonic transform: formula, limits, the gradient bound and serialization;
- the oracle repair: the learner's class, starting exactly at the logger, deterministic; the recoverability tables regenerate byte for byte;
- a one-size run reproduces that size's trials of a multi-size run, which is what lets slices pair with the main runs;
- the worker planner against what a study run holds: dense $\hat{q}$ copies during training and at their peak, and plans for one, two and four GPUs;
- the oracle validation reproduces the Stage 1 winners exactly and extends past a winning edge; the follow-up tables never lower a bound and decompose the target gap exactly.

The world

- calibration targets on the full population; nested levels; seeds
- logger sharpness hits its share exactly; only the logger changes
- popularity column; exact and greedy values on CPU and GPU

Learning and selection

- device scoring equals the numpy reference, CPU and GPU
- weight transforms; DM loss; logit scale; corrections start at the logger
- cross-fitting: folds never see their users, fast forms equal routing

The study

- every arm runs and logs its scores through the study path
- reward-data modes, run keys, CLI defaults, skip rules
- end-to-end reproducibility per seed

Open risks

Found by reading the code; each is a hypothesis for a targeted test. Fixed items are kept, faded, for the record.

■ may change main results ■ post-hoc or robustness ■ reporting

The logger reused its users' random draws (69fffab..c11b2b3, 24 Sep–4 Oct). FIXED `trainer_trials` · `_simulate_from_embedding_policy` · `simulation_utils`

The logger's generator was seeded like the simulation's, and one uniform per row drew both the user and the action. Each user therefore received a nearly fixed action (93–99% of its probability), while the stored propensity was the logger's softmax probability, 6–25 times too small (20–40 at logger share 0.6). The logs had about one effective action per user at every logger share. Fixed in dbc401b: the actions come from their own stream (`derive_seed(..., "logging_policy_actions")`), and the simulation refuses a policy that shares its generator. Every affected experiment was rerun on the corrected logs (`docs/simulator_fix_opc_revalidation_20261004.md`); the regression tests above fail on the old code.

The working defaults are not the paper's choice yet. `custom_losses.py` · `dr_correction` · `docs/training_losses.md` §3.4, §9

Legacy SNDR's objective depends on the batch size. Global SNDR removes that, but it is a stop-gradient, epoch-stale surrogate. DR has no normalizer. Under the log trick, a weight transform optimizes $H(w)$ rather than the named estimate; `--opc-gradient direct` optimizes the estimate itself. All of these are implemented and characterized. Since f5cade9 (27 Sep) the working defaults are DR, the direct gradient and harmonic:0.1; on 4 Oct they were re-tuned on the corrected logs and kept, with a wider learning-rate range. The runs before f5cade9 are reproduced with `--policy-losses sndr --opc-gradient log-trick --train-weights shrink:100`. The paper's method is frozen only at the protocol freeze.

Best hyperparameters carry over between train sizes (TPE). `study.enqueue_trial`

Results at different sizes are not independent, and the order of sizes matters. `--sampler random` removes the warm start for paired comparisons.

Some defaults differ from what the studies used. `run_full_study(_parallel)` · `_run_condition` · `trainer_trials`

The CLI defaults to `--sampler tpe`, which is unpaired and warm-starts each size, so paired studies need `--sampler random`. `--learn-logit-scale` is off, the search space defaults to the older range (the revalidated study passes `--lr-range 1e-4 2e-3 --epochs-range 5 30`), the default seeds are 0/1/2 (development used 100/101), and DM-only and the tempered logger are opt-in. Direct callers of `_run_condition` get an external $\hat{q}$ with no cross-fitting unless they pass `reward_data`, and the trainer API falls back to `shrink:100` weights. Pass these explicitly. The details are in `docs/roee_handoff_20260928.md`.

An oversubscribed GPU slows down silently (WSL2). `memory_budget.py` · `run_full_study_parallel`

Under WSL2, workers that ask for more GPU memory than is free do not fail. The driver spills into shared system memory: the GPU stays 100% busy at about 130 W instead of 250–290 W, every worker slows about 50×, and no error is raised. This was confirmed on 27 Sep with the Windows shared-memory counter (anime at 4 workers). The worker planner now sizes each condition from its own budget. The estimate is the training step at its largest batch + its dense $\hat{q}$ copies + 1.5 GiB. There are 3 dense copies with cross-fitting and a per-size refit, because the next size's lookup is built while the previous one is still held. Each GPU then takes $\text{floor}(0.75 \times \text{free} / \text{peak})$ workers, over any number of cards, with `--max-workers` as the cap (f2e03b5, 05222f8). anime now runs 2 workers on the 48 GB card with no spill. Still open: the planner sizes one run and does not see other runs on the same card. Two concurrent revalidation runs spilled on 4 Oct and were resumed with identical results; keep at most 3 anime workers in total. In GPU mode host RAM is not planned either, and an anime worker holds about 14.5 GiB. On a large multi-GPU machine RAM can bind first. Results are unaffected, because runs are deterministic.

One non-finite gradient failed the whole condition. FIXED `trainer_trials` · `regression_trainer_trial`

Since 4 Oct the trial stops training at the first non-finite gradient, keeps its last finite parameters and is flagged `diverged`; other errors still propagate.

Sharpening is a large part of any stochastic gain.

The tempered logger gains 2.3–6 points over the logger by sharpening alone. With the learnable logit scale OPC sharpens too. With the scale fixed and the older search range (the CausE M5 configuration) OPC is about 1 point lower at 25k on the same logs, and 2.3 lower without bias. Post-hoc tempering on top of the learned scale adds nothing to the selected policy on corrected logs (not adopted). Comparisons with other methods should report greedy values as well, or give every policy the same sharpening.

A logging uniform mix enters the initial reward but not the start point.

Only matters with `--logging-uniform-mix > 0` (off by default).

Labels.

The post-hoc `ipw` column is self-normalized; the trial column `q_error` is the validation standard error.

Fixed this week. FIXED

Clicks capped at the CTR, popularity-dominated vectors and noise that inflated vector length (simulator revamp). A reward model with one ranking for every user (interaction features). Selection clipped at 1. One reward model reused across sizes and fit on extra data (budget-fair $\hat{q}$). No-propensity trials missing re-selection scores (76a29e6). On 27 Sep: the DataLoader's final short batch was upweighted by up to 4.8× (b5efc7d); `--no-log-trick` never reached OPC (documented, 3bed8de); post-hoc estimators used raw weights for an unknown transform, which now raises (d40aaef).

Findings so far

All development runs (seeds 100/101), on the corrected logs since 4 Oct. The numbers and their old-vs-corrected classification are in `docs/simulator_fix_opc_revalidation_20261004.md`; the pre-fix reports are kept as historical records. The CausE comparison (5 Oct) is in `docs/cause_fair_comparison_25k.md`, the BLOB comparison (5 Oct) in `docs/blob_controlled_integration.md`.

Given the same source, BLOB is the weakest model-based learner at 25k.

BLOB-supplied-source (the logger's vectors as BLOB's organic representation, the same 25k rows, no propensities) gains 1.42 greedy points over the logger on the 24 biased worlds. It trails the plain likelihood learner (CausE-cap at $p = 0$) by 1.67 [1.06, 2.29] (higher in 2 of 24 worlds), OPC by 1.27 [0.76, 1.77] and DM-only by 0.74. Without bias it loses least (+0.38 over CausE-cap, 6/6). Its released code reproduces the paper's Table 3 within 0.02 points, so this is not a broken BLOB; it is BLOB applied where its source is wrong.

BLOB adapts little: training, not capacity and not the likelihood principle.

Truth-trained class oracles put BLOB's ranking class within 0.07 points of OPC's structural ceiling, and its infinite-data likelihood fit (+5.49) above the plain learner's (+5.21). About three quarters of its deficit is what its trials reach, the rest NLL selection. Its $K \times K$ deviation stays at about 2.5% of the source term, and it keeps 64% of users on the logger's top item (CausE-cap 38%, OPC 42%). OPC's lead comes from the users it moves to less-logged items (+1.11 of +1.27). Untested lead: the deviation's scale $L = \text{chol}(\Psi^T \Psi / P)$ shrinks as $1/\sqrt{P}$, so at $P = 3.5\text{k} - 10.8\text{k}$ the released ζ prior is much tighter than at the paper's $P = 100$.

The likelihood objective costs value where the mismatch is not a warp.

With unlimited data the likelihood fit in OPC's class reaches its value ceiling under warp (+7.2) but gives up 1.2 (group), 1.1 (vector) and 3.4 (combined) points against it. So the plain likelihood learner's lead over OPC under warp is finite-sample, and the cells where a value objective can matter are group, vector and combined mismatch: the ones the planned richer corrections target.

At equal capacity, CausE's learner ranks better than OPC at 25k, and randomized traffic is not why.

CausE-capacity-matched trains OPC's own correction family on the same source vectors with CausE's objective and NLL selection. It beats OPC by 0.41 greedy points [0.16, 0.66] at $p = 0$ and 0.36 [0.15, 0.57] at $p = 0.25$ (24 biased worlds, paired), with tempered stochastic value close to that. Its lead sits in warp worlds (1.19 points, 6/6). Group and vector are ties. Uniform rows change it by at most 0.05 points, so at $p = 0$ (no randomized rows, no propensities) it is already ahead. It also beats DM-only by 0.94 points. Which part does it, the likelihood objective or the selection rule, is open. 25k only; a targeted 100k check is recommended.

CausE at its native capacity loses, mostly for lack of the source representation.

Started at OPC's source vectors (CausE-warm), CausE gains 8.3 points over native CausE, yet still trails OPC by 1.0 (C) and 2.4 (T): its free per-user vectors learn little from 25k rows (0.15 of the loss repaired; ceiling 1.0). Swapping free vectors for the linear family adds 1.4 more. Randomized traffic is a pure cost at 25k: $p = 0.25$ gives up 932 clicks per world (21% of the collection's clicks) for no gain in either fair variant.

Structure decides what a linear repair can fix.

We trained the learner's own linear class on the true click model. It recovers 0.99 of the ranking loss from warp, 0.56–0.78 from group and 0.31–0.61 from vector (high level). The order warp > group > vector held in all 6 dataset × seed pairs, and a validation with up to 9× the optimization budget moved it by at most 0.025. This uses no logged data, so the logging fix leaves it unchanged.

Logged learning recovers a fifth to a half of it.

OPC reaches 0.21 / 0.38 / 0.49 of the oracle's ranking repair at 5k / 25k / 100k (biased worlds), 5.6–7.5 points over the logger. At 100k the loss splits (structural / not learned / learned) as warp 0.01 / 0.47 / 0.52, group 0.32 / 0.39 / 0.29, vector 0.50 / 0.29 / 0.21 and combined about 0.32 / 0.30 / 0.37. No-propensity learning recovers 0.10–0.15, and sharpening alone none.

Propensities add value over a good reward model.

OPC beats DM-only by +1.22 / +0.64 / +0.77 points at 5k / 25k / 100k (biased worlds), and for every single-type bias at every size with every interval above 0. Under combined high bias the two tie below 100k; there DM-only leads on ml at 25k by 1.0 point. OPC beats no-propensity by 1.9–2.4 points from 25k.

Unneeded correction costs a little.

With no bias OPC trails sharpening alone (the tempered logger) by 0.6–1.2 points, and at 100k no-propensity by 0.4.

OPC holds up when the reward model's data shrink.

Cutting $\hat{q}$'s data from a separate 50k-row slice to the policy's own rows costs DM-only 2.15 points at 5k and OPC 0.25 (within noise). With 50k rows for $\hat{q}$, DM-only beats OPC at 5k by 1.2.

A misspecified reward model breaks DM-only, and OPC's capped weights at large n.

With concat features (one ranking for every user), DM-only loses 3.6–5.6 points: $\hat{q}$ overrates its pick by 11–17 points and its policy leaves the logged support. OPC loses 1.6 points at 25k and 4.5 at 100k with harmonic:0.1 or shrink:100: their cap at 10 lets $\hat{q}$'s errors steer training off the support, where the clipped selection estimate cannot see it. With raw training weights OPC loses nothing at 100k, at a cost of 0.8 points when $\hat{q}$ is well specified. OPC still beats DM-only.

Logger support 0.6–0.95 does not remove recoverability (25k).

The corrected loggers spread over 31–36, 7–8 and 2–3 effective actions per user at shares 0.6, 0.8 and 0.95. OPC's recovery is flat from 0.8 to 0.95, slightly lower at 0.6 for warp and group (not significant), and OPC leads DM-only at every share. A logger without full support was not tested.

The weighting defaults survive re-tuning.

On corrected logs every regularized weighting beats raw DR, the best amount of regularization falls with n, harmonic $\lambda = 0.1$ –0.3 lead, and Su's shrink:100 is within 0.2 points of harmonic:0.1 in every cell of the Stage 2 grid. Exact SNDR adds nothing over DR, and the log trick never beats the direct gradient. The learning-rate range was too narrow and is now $1e-4$ – $2e-3$, with 5–30 epochs. The weighting is regime dependent: with a misspecified reward model raw DR is the robust choice (above).

Selection works; training is the bottleneck.

The clip:10 DR lower bound picks within 0.03–0.11 points of the best of 20 trials, and OPC's selection regret is 0.09–0.14 points. Its DR estimate of the selected policy is unbiased within noise from 25k (it was optimistic on the buggy logs). DM-only's reward model overrates its own pick, by 5.3 points at 5k.

Plan

Status, 4 Oct

DONE A logging bug (69fffab..c11b2b3: each user's action reused the draw that picked the user) was found, fixed (dbc401b) and made a tested invariant. Every affected experiment was rerun on the corrected logs: Stage 2 with all arms and the shrink:100 robustness arm, the logging-support sweep, both reward-model tests, and a decomposition of the simulator fix vs the retuning. The OPC defaults were re-tuned on new development seeds (200/201): DR, the direct gradient, harmonic:0.1 and clip:10 selection are kept, and the learning-rate range is widened to $1e-4$ – $2e-3$ with 5–30 epochs. Every old finding is classified old vs corrected in `docs/simulator_fix_opc_revalidation_20261004.md`; figures and tables are in `artifacts/full_study/opc_revalidation_20261004/report/`. The earlier reports are kept as historical records, marked as produced on the buggy simulator.

CausE, 5 Oct

DONE The fair 25k comparison on branch `cause-fair` (archive `origin/cause-baseline = d31a5dd`). Native CausE (M5 rows, reused), CausE-warm and CausE-capacity-matched, at $p \in \{0, .01, .05, .10, .15, .25\}$, against the revalidated OPC. The search spaces were tuned on seeds 200/201 by a pre-registered rule. **AWAITING REVIEW** The next decision is the user's: a targeted 100k run (CausE-cap at $p \in \{0, .05, .25\}$), and whether to check OPC's objective vs selection against the in-class likelihood learner.

BLOB, 5 Oct

DONE The released code reproduces the paper's Table 3; the PyTorch port replays the released graph step by step. BLOB-supplied-source was compared at 25k on the 30 development worlds with the reused OPC, CausE-cap, CausE-warm and DM-only rows, on branch `blob-controlled-integration`. Its search space came from a pre-registered rule on seeds 200/201, with one supplementary round. **AWAITING REVIEW** Group / regional / per-vector correction, new bias mechanisms, propensity-weighted likelihood and RecoGym wait for the review.

Dev grid

LATER linear vs linear+MLP vs MLP corrections for no bias, medium, high, and warp, group or vector alone (ml, kuairand, anime).

Final report

THEN all six datasets $\times$ no / low / medium / high bias $\times$ warp-only, group-only, vector-only $\times$ 5k, 25k, 100k, four arms, fresh seeds.

Model changes

PENDING bilinear reward model; per-group and per-item offsets with partial pooling; popularity runs; cluster-level weights for large catalogs; Switch-DR; validation as a share of the budget; aligning H1 with the new defaults; real-data checks (Open Bandit Dataset, KuaiRec's full matrix, KuaiRand's random exposures, Coat, Yahoo! R3).

Assumption

Logged propensities are known exactly, as in most OPE and OPL work; estimating them is outside this study. Since 4 Oct the simulator's logged propensities are tested to be the probabilities that generated each row.

Deferred

Comparing policies by their exploitation (greedy) part alone. The greedy value is logged on every trial for when we return to it.